1 Tests

Cette partie vérifie que `gaussp` calcule le même résultat que le programme séquentiel `gauss`, mesure ses performances sur une machine puis sur plusieurs, et étudie son comportement sur des entrées incorrectes. Toutes les mesures ont été faites le 23 octobre 2026. Les commandes exactes et leurs sorties sont conservées dans `tests/logs/`, les captures dans les dossiers `captures_*`.

1.1 Conditions des essais

Machine principale	`pc5201l` : Intel Core i7-12700 (12 cœurs, 20 threads), 31 Gio de mémoire
Autres machines	`pc5201a`, `pc5201b`, `pc5201c` (même architecture `LINUX64`)
Système	Debian 13, noyau 6.12.113, gcc 14.2.0, PVM 3.4.6
Réseau	Ethernet 1 Gbit/s ; latence vers `pc5201a` et `pc5201c` : 0,8 ms en moyenne
Compilation	`gauss` et `gaussp` avec le même Makefile, sans optimisation

Tableau 1. – Environnement de test.

- Le chronométrage de `gaussp` sépare le chargement et le calcul ; la sauvegarde n'est mesurée que dans un test dédié.
- Dans la version de départ, l'appel `gauss(tab, N, P, me, tids)` était commenté dans `dowork()` (étape de test du chargement). Il a été réactivé pour tous les tests de calcul.
- Les tests qui demandaient une version modifiée (sans calcul, avec affichage ou chronométrage supplémentaire) utilisent une copie temporaire de `gaussp.c`, créée puis supprimée par le script ; `gaussp.c` n'est pas modifié.

Matrices. `m4`, `m16`, `m500`, `m1000`, `m5000` et `m10000` sont les matrices fournies (`/nfs/opt/pvm/pvm3`), avec les résultats de référence `m4.triangule`, `m16.triangule` et `m500.triangule`. Aucune `m8` n'étant fournie, celle du dépôt du projet est utilisée. Les matrices d'erreur viennent du dépôt (`matrices-test/erreurs/`).

1.2 Validation du calcul

1.2.1 Test de chargement et de sauvegarde

Avec l'appel à `gauss()` commenté, `gaussp` doit réécrire exactement la matrice lue. La comparaison `cmpm` vérifie l'égalité valeur par valeur.

Matrice	P testés	Résultat de `cmpm`
`m4`	1, 2, 4	16 valeurs, 0 différence
`m8`	1, 2, 4, 8	64 valeurs, 0 différence
`m16`	1, 2, 4, 8, 16	256 valeurs, 0 différence
`m500`	1, 2, 4, 5, 10, 20	250 000 valeurs, 0 différence
`m1000`	1, 2, 4, 8, 10, 20	1 000 000 valeurs, 0 différence

Tableau 2. – Aller-retour chargement / sauvegarde : 24 essais, tous identiques.

Ce test ne prouve pas à lui seul que chaque tâche reçoit les bonnes lignes : deux erreurs inverses pourraient se compenser. Nous avons donc utilisé une matrice dont la ligne i ne contient que la valeur i, et une variante où chaque tâche affiche les lignes reçues. Pour `m8` (P = 4) et `m16` (P = 4 et 8), toutes les lignes sont au bon endroit : la tâche `me` détient les lignes `me`, `me + P`, `me + 2P`… (0 erreur sur 8, 16 et 16 lignes).

```
safarl@pc5201l:~/pvm3/matrices-test$ cat ../tests/tmp_mat/m16-lignes
    0.00     0.00     0.00     0.00     0.00     0.00     0.00     0.00     0.00     0.00     0.00     0.00     0.00     0.00     0.00     0.00
    1.00     1.00     1.00     1.00     1.00     1.00     1.00     1.00     1.00     1.00     1.00     1.00     1.00     1.00     1.00     1.00
    2.00     2.00     2.00     2.00     2.00     2.00     2.00     2.00     2.00     2.00     2.00     2.00     2.00     2.00     2.00     2.00
    3.00     3.00     3.00     3.00     3.00     3.00     3.00     3.00     3.00     3.00     3.00     3.00     3.00     3.00     3.00     3.00
    4.00     4.00     4.00     4.00     4.00     4.00     4.00     4.00     4.00     4.00     4.00     4.00     4.00     4.00     4.00     4.00
    5.00     5.00     5.00     5.00     5.00     5.00     5.00     5.00     5.00     5.00     5.00     5.00     5.00     5.00     5.00     5.00
    6.00     6.00     6.00     6.00     6.00     6.00     6.00     6.00     6.00     6.00     6.00     6.00     6.00     6.00     6.00     6.00
    7.00     7.00     7.00     7.00     7.00     7.00     7.00     7.00     7.00     7.00     7.00     7.00     7.00     7.00     7.00     7.00
    8.00     8.00     8.00     8.00     8.00     8.00     8.00     8.00     8.00     8.00     8.00     8.00     8.00     8.00     8.00     8.00
    9.00     9.00     9.00     9.00     9.00     9.00     9.00     9.00     9.00     9.00     9.00     9.00     9.00     9.00     9.00     9.00
   10.00    10.00    10.00    10.00    10.00    10.00    10.00    10.00    10.00    10.00    10.00    10.00    10.00    10.00    10.00    10.00
   11.00    11.00    11.00    11.00    11.00    11.00    11.00    11.00    11.00    11.00    11.00    11.00    11.00    11.00    11.00    11.00
   12.00    12.00    12.00    12.00    12.00    12.00    12.00    12.00    12.00    12.00    12.00    12.00    12.00    12.00    12.00    12.00
   13.00    13.00    13.00    13.00    13.00    13.00    13.00    13.00    13.00    13.00    13.00    13.00    13.00    13.00    13.00    13.00
   14.00    14.00    14.00    14.00    14.00    14.00    14.00    14.00    14.00    14.00    14.00    14.00    14.00    14.00    14.00    14.00
   15.00    15.00    15.00    15.00    15.00    15.00    15.00    15.00    15.00    15.00    15.00    15.00    15.00    15.00    15.00    15.00
safarl@pc5201l:~/pvm3/matrices-test$ ../tests/pvmrun.py 60 "spawn -4 -> gaussp_dist 16 /user/safarl/pvm3/tests/tmp_mat/m16-lignes" | grep -v '^pvm> $'
 | tee /tmp/dist.txt
pvmd already running.
pvm> spawn -4 -> gaussp_dist 16 /user/safarl/pvm3/tests/tmp_mat/m16-lignes
[1]
4 successful
t40338
t40339
t4033a
t4033b
pvm> [1:t4033a] tache 2 : ligne locale 0 -> ligne 2 de la matrice (attendu 2)
[1:t4033a] tache 2 : ligne locale 1 -> ligne 6 de la matrice (attendu 6)
[1:t4033a] tache 2 : ligne locale 2 -> ligne 10 de la matrice (attendu 10)
[1:t4033a] tache 2 : ligne locale 3 -> ligne 14 de la matrice (attendu 14)
[1:t40338] tache 3 : ligne locale 0 -> ligne 3 de la matrice (attendu 3)
[1:t40338] tache 3 : ligne locale 1 -> ligne 7 de la matrice (attendu 7)
[1:t40338] tache 3 : ligne locale 2 -> ligne 11 de la matrice (attendu 11)
[1:t40338] tache 3 : ligne locale 3 -> ligne 15 de la matrice (attendu 15)
[1:t40339] tache 0 : ligne locale 0 -> ligne 0 de la matrice (attendu 0)
[1:t40339] tache 0 : ligne locale 1 -> ligne 4 de la matrice (attendu 4)
[1:t40339] tache 0 : ligne locale 2 -> ligne 8 de la matrice (attendu 8)
[1:t40339] tache 0 : ligne locale 3 -> ligne 12 de la matrice (attendu 12)
[1:t40339] Loading time: 0.000864 sec.
[1:t40339] Computation time: 0.000386 sec.
[1:t4033b] tache 1 : ligne locale 0 -> ligne 1 de la matrice (attendu 1)
[1:t4033b] tache 1 : ligne locale 1 -> ligne 5 de la matrice (attendu 5)
[1:t4033b] tache 1 : ligne locale 2 -> ligne 9 de la matrice (attendu 9)
[1:t4033b] tache 1 : ligne locale 3 -> ligne 13 de la matrice (attendu 13)
[1:t40338] EOF
[1:t4033a] EOF
[1:t40339] EOF
[1:t4033b] EOF
[1] finished

pvm> quit

Console: exit handler called
pvmd still running.
safarl@pc5201l:~/pvm3/matrices-test$ grep -o 'tache .*' /tmp/dist.txt | sort -k2,2n -k6,6n
tache 0 : ligne locale 0 -> ligne 0 de la matrice (attendu 0)
tache 0 : ligne locale 1 -> ligne 4 de la matrice (attendu 4)
tache 0 : ligne locale 2 -> ligne 8 de la matrice (attendu 8)
tache 0 : ligne locale 3 -> ligne 12 de la matrice (attendu 12)
tache 1 : ligne locale 0 -> ligne 1 de la matrice (attendu 1)
tache 1 : ligne locale 1 -> ligne 5 de la matrice (attendu 5)
tache 1 : ligne locale 2 -> ligne 9 de la matrice (attendu 9)
tache 1 : ligne locale 3 -> ligne 13 de la matrice (attendu 13)
tache 2 : ligne locale 0 -> ligne 2 de la matrice (attendu 2)
tache 2 : ligne locale 1 -> ligne 6 de la matrice (attendu 6)
tache 2 : ligne locale 2 -> ligne 10 de la matrice (attendu 10)
tache 2 : ligne locale 3 -> ligne 14 de la matrice (attendu 14)
tache 3 : ligne locale 0 -> ligne 3 de la matrice (attendu 3)
tache 3 : ligne locale 1 -> ligne 7 de la matrice (attendu 7)
tache 3 : ligne locale 2 -> ligne 11 de la matrice (attendu 11)
tache 3 : ligne locale 3 -> ligne 15 de la matrice (attendu 15)
safarl@pc5201l:~/pvm3/matrices-test$ grep -o 'tache .*' /tmp/dist.txt | awk '{n++; v=$9; a=$14; sub(/\)/,"",a); if (v!=a || /MELANGEE/) e++} END {prin
t n" lignes vérifiées, "e+0" erreurs de distribution"}'
16 lignes vérifiées, 0 erreurs de distribution
safarl@pc5201l:~/pvm3/matrices-test$ cmpm ../tests/tmp_mat/m16-lignes ../tests/tmp_mat/m16-lignes.result
256 valeurs, 0 différences
```

Fig. 1. – Distribution des lignes de `m16` sur 4 tâches.

1.2.2 Comparaison avec le séquentiel et la référence

Le script `quantif.py` vérifie que tous les coefficients sous la diagonale sont nuls et compare le résultat au séquentiel et au `.triangule` fourni.

Matrice	P	Nuls sous la diagonale	Écart / séquentiel	Écart / `.triangule`
`m4`	2	6 / 6	0	0
`m8`	4	28 / 28	0	—
`m16`	4	120 / 120	0	0
`m500`	4	124 750 / 124 750	0	0
`m1000`	4	499 500 / 499 500	0	—
`m5000`	4	12 497 500 / 12 497 500	0	—

Tableau 3. – Quantification des matrices résultats (écart maximal en valeur absolue).

D'autres comparaisons ont été faites au cours des campagnes : `m4` (P = 1, 2, 4), `m16` (P = 1, 2, 4, 8), `m500` (P = 2, 4, 10), `m1000` (P = 4, 8), `m5000` et `m10000` sur une machine et sur 2 ou 4 machines. Tous les résultats sont identiques au séquentiel. Ces vérifications portent sur les valeurs écrites avec deux décimales.

```
safarl@pc5201l:~/pvm3/matrices-test$ rm -f m8.result
safarl@pc5201l:~/pvm3/matrices-test$ ../tests/pvmrun.py 60 "spawn -4 -> gaussp 8 $PWD/m8" | grep -v '^pvm> $'
pvmd already running.
pvm> spawn -4 -> gaussp 8 /user/safarl/pvm3/matrices-test/m8
[1]
4 successful
t4014b
t4014c
t4014d
t4014e
pvm> [1:t4014d] Loading time: 0.006074 sec.
[1:t4014d] Computation time: 0.000445 sec.
[1:t4014c] EOF
[1:t4014d] EOF
[1:t4014b] EOF
[1:t4014e] EOF
[1] finished

pvm> quit

Console: exit handler called
pvmd still running.
safarl@pc5201l:~/pvm3/matrices-test$ echo "===== matrice d'origine : m8"; cat m8
===== matrice d'origine : m8
161.80 6.44 -7.68 0.90 -7.31 -1.86 4.77 0.14
7.12 168.98 -0.90 -0.52 -0.05 -5.67 -6.98 -4.94
0.91 1.34 159.11 -1.35 -1.61 4.92 1.86 7.83
-1.25 -3.62 -5.18 160.85 -7.56 -3.77 9.75 -2.43
-5.05 -1.53 0.69 6.21 154.21 0.74 2.48 -3.22
2.51 -0.94 6.62 -0.69 -5.66 165.75 0.03 -3.81
-3.30 -8.82 5.06 -9.49 -2.02 6.25 165.77 9.71
3.64 5.65 -2.47 2.47 3.04 1.13 4.50 154.61
safarl@pc5201l:~/pvm3/matrices-test$ echo "===== matrice résultat (triangulaire) : m8.result"; cat m8.result
===== matrice résultat (triangulaire) : m8.result
  161.80     6.44    -7.68     0.90    -7.31    -1.86     4.77     0.14
    0.00   168.70    -0.56    -0.56     0.27    -5.59    -7.19    -4.95
    0.00     0.00   159.16    -1.35    -1.57     4.97     1.89     7.87
    0.00     0.00     0.00   160.80    -7.66    -3.74     9.70    -2.27
    0.00     0.00     0.00    -0.00   154.29     0.77     2.19    -3.19
    0.00     0.00     0.00     0.00     0.00   165.55    -0.05    -4.30
    0.00     0.00     0.00     0.00     0.00     0.00   166.05     9.18
    0.00     0.00     0.00     0.00     0.00     0.00     0.00   154.78
safarl@pc5201l:~/pvm3/matrices-test$ ../tests/quantif.py m8.result 8 m8.seq.result
valeurs=64/64 | sous la diagonale : 28/28 nulles, max |a_ij|=0.00 | plus petit pivot |a_kk|=154.29 | vs m8.seq.result : 0 écarts > 0.011, écart max=0.
00
```

Fig. 2. – `m8` avec P = 4 : matrice d'origine, matrice triangulaire obtenue, quantification.

1.2.3 Observation avec XPVM

`gaussp` a été lancé depuis XPVM avec 2 et 4 tâches sur `m4`, `m8` et `m16`, réparties sur `pc5201l`, `pc5201a` et `pc5201c`. Les 6 essais donnent le même résultat que le séquentiel.

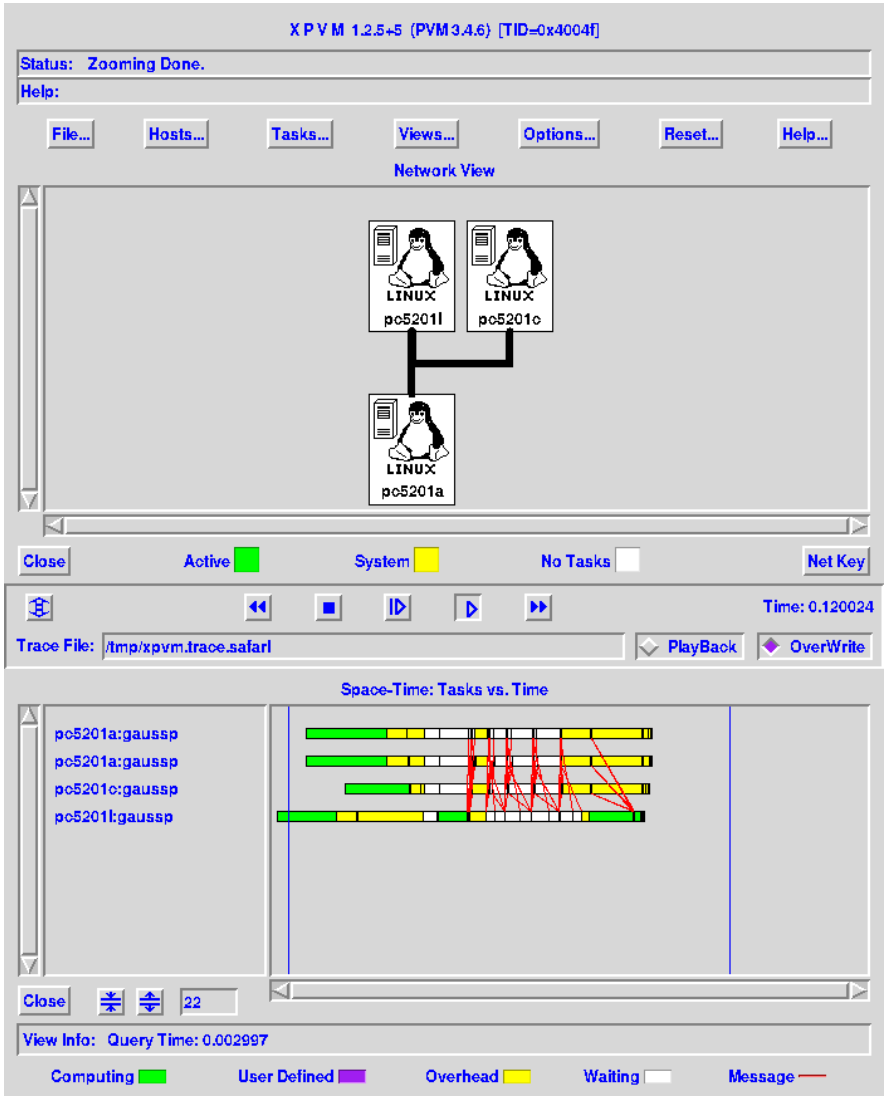


Fig. 3. – Vue Space-Time de XPVM, `m16` avec P = 4.

La trace montre la lecture du fichier par P0 pendant que les autres tâches attendent, puis la distribution des lignes, les diffusions successives des lignes pivots (messages rouges) et le rassemblement final vers P0. Sur une matrice aussi petite, les communications et les synchronisations occupent presque tout le temps.

1.3 Mesures de performance

Le temps de calcul inclut les échanges de lignes pivots et la barrière finale. On note $S(P) = \frac{T_{\text{seq}}}{T_{\text{par}}}$ et $E = \frac{S}{P}$. Les temps sont des médianes : 5 essais pour N = 500 et 1 000, 1 essai pour 5 000 et 10 000. Le temps séquentiel de `m10000` (627,2 s) provient d'une exécution manuelle sur la même machine le même jour.

1.3.1 Une seule machine

Version	N = 500	N = 1 000	N = 5 000	N = 10 000
Séquentiel	0,0836 s	0,672 s	80,2 s	627,2 s
P = 1	0,0745 (1,12)	0,596 (1,13)	—	—
P = 2	0,0501 (1,67)	0,376 (1,78)	—	—
P = 4	0,0332 (**2,52**)	0,225 (2,98)	19,4 (4,13)	150,9 (4,16)
P = 8	—	0,184 (**3,65**)	—	—
P = 10	0,0356 (2,35)	0,185 (3,64)	14,7 (5,47)	115,7 (**5,42**)
P = 20	0,0570 (1,47)	0,239 (2,81)	14,0 (**5,75**)	—

Tableau 4. – Temps de calcul (s) et speedup entre parenthèses, sur `pc5201l`.

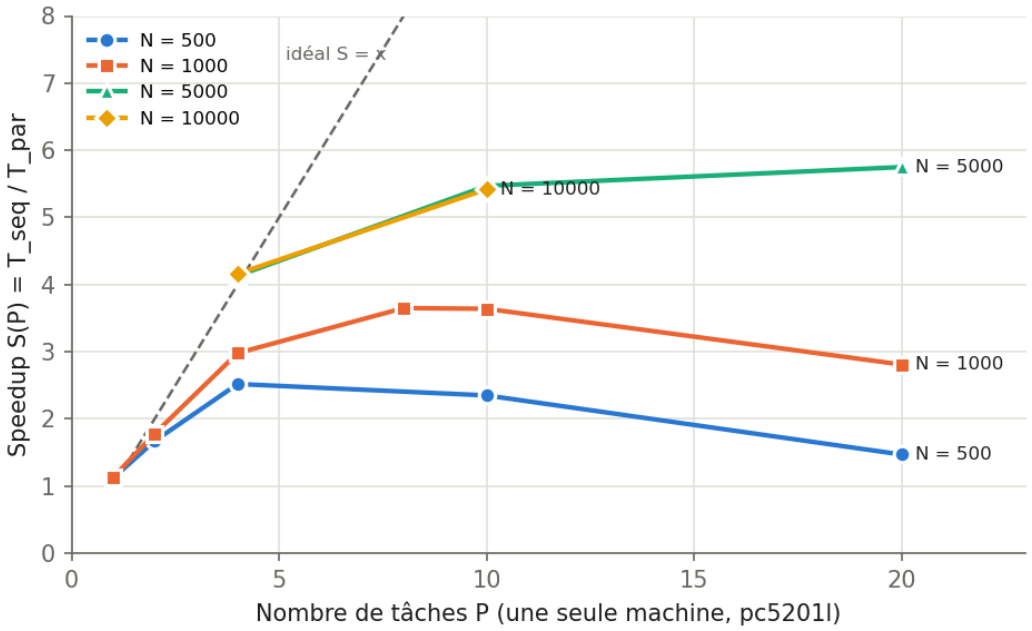


Fig. 4. – Speedup en fonction du nombre de tâches sur une machine.

- Le gain augmente avec la taille : le meilleur speedup passe de 2,52 (N = 500) à 3,65 (N = 1 000) puis environ 5,5 à 5,75 (N = 5 000 et 10 000).
- Pour les petites matrices, le gain plafonne vite puis se dégrade : avec P = 20, N = 500 est plus lent qu'avec P = 4.
- P = 1 est environ 12 % plus rapide que le séquentiel sur toutes les tailles. Ce gain ne vient pas du parallélisme ; il vient probablement d'une écriture différente des boucles et des calculs d'indice, ce que les mesures seules ne permettent pas de démontrer.

1.3.2 Plusieurs machines

Le nombre de machines varie de 4 à 2 puis 1 (hôtes retirés avec `delete`). PVM répartit les tâches de façon équilibrée, ce que confirment les identifiants des tâches dans le journal. Deux séries : 1 tâche par machine (P = nombre de machines) et 5 tâches par machine.

1 tâche / machine	**N = 500**	**N = 1 000**	**N = 5 000**	**N = 10 000**
1 machine	1,11 (111 %)	1,13 (113 %)	1,15 (115 %)	—
2 machines	0,32 (16 %)	0,60 (30 %)	1,52 (76 %)	1,83 (92 %)
4 machines	0,25 (6 %)	0,49 (12 %)	1,99 (50 %)	**2,85 (71 %)**
5 tâches / machine				
1 machine (P = 5)	2,92	3,65	4,86	**4,80**
2 machines (P = 10)	0,10	0,21	1,09	—
4 machines (P = 20)	0,09	0,21	0,93	1,88

Tableau 5. – Speedup (et efficacité par machine) selon le nombre de machines. « — » : essai non réalisé.

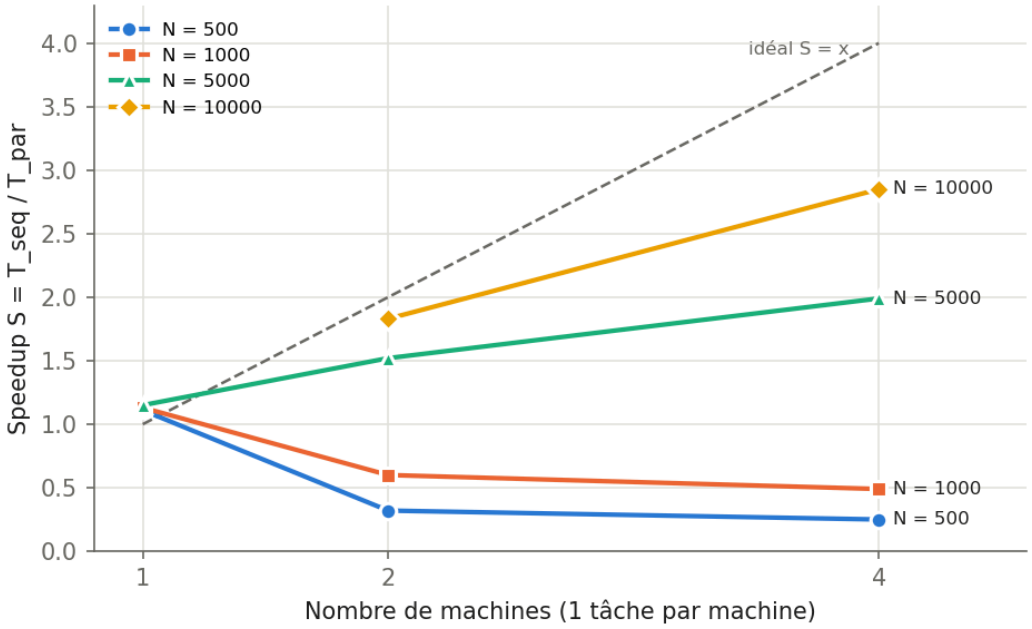


Fig. 5. – Speedup en fonction du nombre de machines, 1 tâche par machine.

- Répartir le calcul sur plusieurs machines n'est utile que pour les grandes matrices : pour N = 500 et 1 000, la version répartie est plus lente que le séquentiel. Pour N = 10 000, 4 machines donnent un speedup de 2,85.
- L'efficacité augmente avec N : sur 2 machines, de 16 % (N = 500) à 92 % (N = 10 000). Chaque tâche calcule davantage entre deux réceptions de pivot.
- Une machine avec 5 tâches fait mieux que 4 machines avec une tâche chacune (4,80 contre 2,85 pour N = 10 000), et multiplier les tâches sur plusieurs machines dégrade fortement les performances (1,88 avec P = 20 sur 4 machines). Une explication plausible est le coût du réseau : chaque ligne pivot est envoyée à P − 1 tâches, et ces envois passent par les démons PVM et l'Ethernet dès que les tâches ne sont pas sur la même machine.

1.3.3 Chargement et sauvegarde

	Séquentiel	**P = 1**	**P = 2**	**P = 4**	**P = 8**
Chargement (s)					
`m500`	0,039	0,032	0,028	0,026	—
`m1000`	0,105	0,105	0,098	0,106	0,100
`m5000`	2,23	2,26	2,34	2,40	2,41
Sauvegarde (s)					
`m500`	0,092	0,096	0,098	0,105	—
`m1000`	0,277	0,287	0,291	0,288	0,295
`m5000`	4,91	4,79	5,08	5,05	5,11

Tableau 6. – Médianes de 3 essais sur `pc5201l` (5 pour le chargement séquentiel).

Le chargement et la sauvegarde ne s'accélèrent pas quand P augmente : P0 lit et écrit tout le fichier seul, et doit en plus envoyer puis recevoir les lignes des autres tâches. Pour `m5000`, les deux augmentent légèrement avec P (de 2,26 à 2,41 s et de 4,79 à 5,11 s). Pour `m500` et `m1000`, les écarts sont de l'ordre du bruit de mesure. Le premier essai de chaque série était plus lent (fichier pas encore en cache), d'où l'usage de la médiane.

Pour `m5000` sur 20 tâches, chargement et sauvegarde (environ 7,5 s) pèsent donc de plus en plus lourd devant un calcul réduit à 14 s.

1.4 Tests des cas d'erreur

Chaque cas est lancé dans la console PVM. Après 15 s, `ps -a` montre les tâches restantes, supprimées ensuite avec `reset`.

Cas	P	Message de `gaussp`	Bloquées	`.result`
Fichier inexistant	4	`matrix_load : fopen: No such file or directory`	3	non
Fichier incomplet (14/16)	2	aucun, se termine	0	faux
Coefficient non numérique	2	`pivot 2 presque nul` (par hasard)	1	non
Pivot nul au départ	2	`ATTENTION: pivot 0 presque nul`	1	non
Pivot nul pendant le calcul	2	`ATTENTION: pivot 1 presque nul`	1	non
N = 6 non divisible par P = 4	4	aucun	4	vide
N = 0	2	aucun, se termine	0	vide
N = −4	2	aucun, se termine	0	vide
Arguments manquants	2	aucun, les tâches plantent	0	non
Sortie non inscriptible	2	`matrix_save : fopen: Permission denied`	0	non

Tableau 7. – Comportement observé de `gaussp` sur des entrées incorrectes.

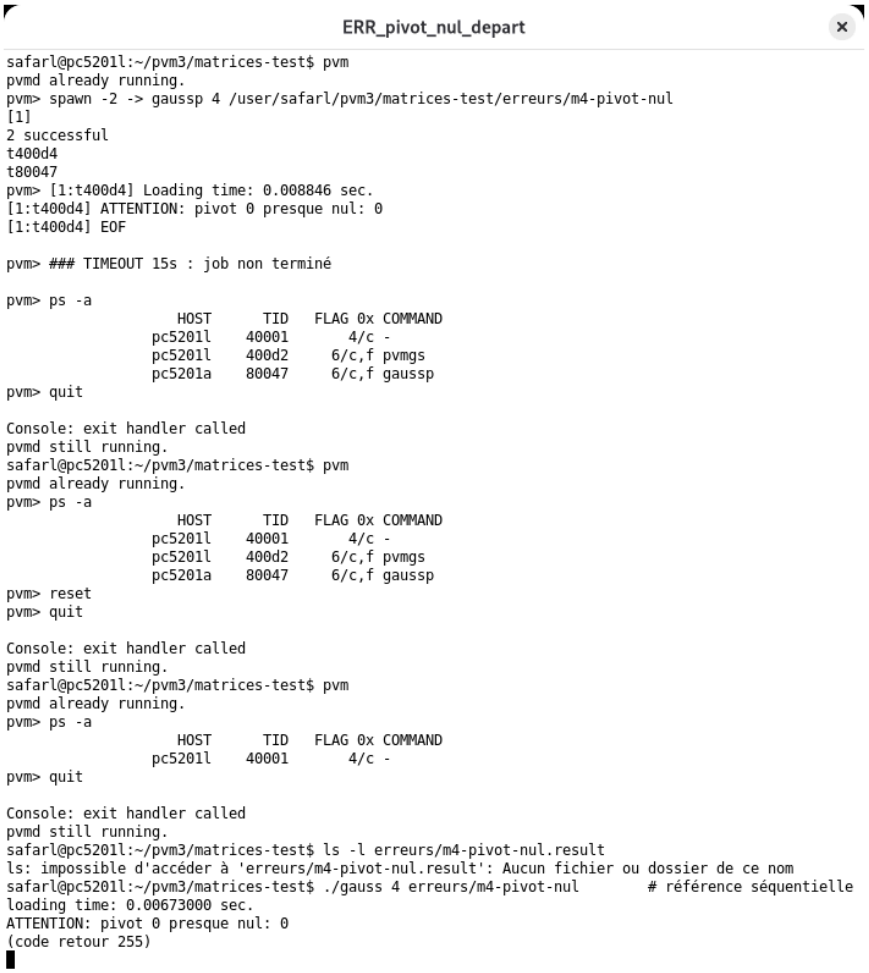


Fig. 6. – Pivot nul : P0 affiche le message et s'arrête, l'autre tâche reste bloquée.

- Quand P0 s'arrête avec `exit()`, les autres tâches restent bloquées dans `pvm_recv` ou `pvm_barrier` et doivent être tuées à la main.
- La valeur de retour de `fscanf` n'est pas vérifiée : un fichier incomplet donne un résultat faux sans message, et un coefficient non numérique n'est détecté que par chance, via un pivot nul.

- N n'est pas contrôlé : non divisible par P, tout se bloque ; nul ou négatif, le programme se termine sans message avec un fichier vide.
- Le séquentiel `gauss` a les mêmes faiblesses et plante même (erreur de segmentation) si le fichier est absent ou la sortie non inscriptible.

Corrections possibles : vérifier `argc`, `N > 0` et `N % P == 0` avant tout calcul, tester le retour de `fopen` et de `fscanf`, et en cas d'erreur prévenir les autres tâches (message dédié ou `pvm_kill`) au lieu d'un simple `exit()`.

Sur plusieurs machines, deux précautions ont été nécessaires pendant les essais : `reset` met quelques secondes à tuer les tâches distantes, et il tue aussi le serveur de groupes `pvmgs`, qu'il vaut mieux relancer avant l'essai suivant.